

Scientific Computing Hw3

Chao-Yang Zhan

March 31, 2026

Exercise 3.1 (*Poisson problem on Rectangular Domains*)

In this exercise, we develop a numerical solver for the two-dimensional Poisson equation on a general rectangular domain $\Omega = [a_x, b_x] \times [a_y, b_y]$:

$$\nabla^2 u(x, y) = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \quad (1)$$

subject to Dirichlet boundary conditions $u(x, y) = g(x, y)$ on $\partial\Omega$. For testing purposes, we use $f(x, y) = 1.25 \exp(x + y/2)$, which corresponds to the exact solution $u_{true}(x, y) = \exp(x + y/2)$.

(c) Generalization to $\Delta x \neq \Delta y$ on Rectangular Domains

We generalize the 5-point Laplacian solver to handle different grid spacings $\Delta x = h_x$ and $\Delta y = h_y$. Using second-order central differences, the discretized equation at each interior point (x_i, y_j) is:

$$\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} + \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} = f_{i,j} \quad (2)$$

The system $Au = F$ is constructed using Kronecker products:

$$A = \frac{1}{h_x^2}(I_{m_y} \otimes T_{m_x}) + \frac{1}{h_y^2}(S_{m_y} \otimes I_{m_x}) \quad (3)$$

where the 1D tridiagonal matrices T_{m_x} and S_{m_y} are defined as:

$$T_{m_x} = \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \cdots & 1 & -2 & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{pmatrix}, \quad S_{m_y} = \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \cdots & 1 & -2 & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{pmatrix} \quad (4)$$

MATLAB Implementation:

Below is the MATLAB script used to solve the Poisson problem on the rectangular domain $[0, 2] \times [0, 1]$ with $m_x = 20$ and $m_y = 40$.

```

clear; clc; format long g;

% Domain and Mesh Definition
x1=0; x2=1; y1=0; y2=2;
m_x=20; m_y=40;
h_x=(x2-x1)/(m_x+1);
h_y=(y2-y1)/(m_y+1);

x=linspace(x1,x2,m_x+2);
y=linspace(y1,y2,m_y+2);
[X,Y]=meshgrid(x,y);
X=X'; Y=Y';

% Interior indices
Iint = 2:m_x+1; Jint = 2:m_y+1;
Xint = X(Iint,Jint); Yint = Y(Iint,Jint);

% RHS and Exact Solution
f = @(x,y) 1.25*exp(x+y/2);
rhs = f(Xint,Yint);
u_true = exp(X+Y/2);
u_soln = u_true;

% Boundary condition adjustments
rhs(:,1) = rhs(:,1) - u_soln(Iint,1)/(h_y)^2;
rhs(:,m_y) = rhs(:,m_y) - u_soln(Iint,m_y+2)/(h_y)^2;
rhs(1,:) = rhs(1,:) - u_soln(1,Jint)/(h_x)^2;
rhs(m_x,:) = rhs(m_x,:) - u_soln(m_x+2,Jint)/(h_x)^2;

% Matrix Construction via Kronecker Products
F = reshape(rhs,m_x*m_y,1);
I_x = speye(m_x); I_y = speye(m_y);
e_x = ones(m_x,1);
T_x = spdiags([e_x -2*e_x e_x],[-1 0 1],m_x,m_x);
e_y = ones(m_y,1);
S_y = spdiags([e_y -2*e_y e_y],[-1 0 1],m_y,m_y);

A = kron(I_y,T_x)/(h_x)^2 + kron(S_y,I_x)/(h_y)^2;

% Solver
u_vec = A \ F;
u_soln(Iint, Jint) = reshape(u_vec, m_x, m_y);

% Error
error_grid = abs(u_soln - u_true);
max_err = max(error_grid(:));
fprintf('Max Error = %e\n', max_err);

```

```

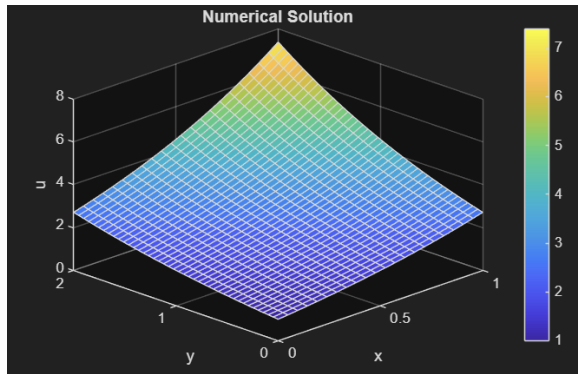
% plot(Numerical Solution)
figure(1); clf;
surf(X, Y, u_soln);
colorbar;
xlabel('x'); ylabel('y'); zlabel('u');
title(['Numerical Solution']);
view(-45, 30);
grid on;
drawnow;

% plot(Error Surface)
figure(2); clf;
surf(X, Y, error_grid);
colorbar;
axis tight;
xlabel('x'); ylabel('y'); zlabel('|Error|');
title(['Absolute Error Distribution']);
grid on;
view(-45, 30);
drawnow;

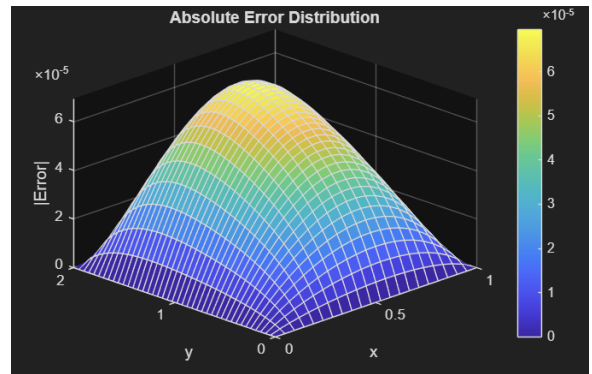
```

Numerical Results and Analysis:

The figures below illustrate the performance of the generalized solver on the rectangular domain $[0, 1] \times [0, 2]$ with $m_x = 20$ and $m_y = 40$, resulting in $h_x \approx 0.0476$ and $h_y \approx 0.0488$.



(a) Numerical Solution $u(x, y)$



(b) Absolute Error Distribution

Figure 1: Numerical results for part (c). The surface plot (left) represents the computed solution, while the error plot (right) shows the discrepancy $|u_{soln} - u_{true}|$.

Observations:

- **Solution Accuracy:** As shown in Figure (a), the numerical solution smoothly approximates the exact exponential function $e^{x+y/2}$ across the rectangular domain.
- **Error Distribution:** Figure (b) demonstrates that the absolute error is zero along all four boundaries, which is expected due to the imposition of Dirichlet boundary conditions.

- **Maximum Discrepancy:** The error reaches its maximum value of approximately 6.95×10^{-5} near the center of the domain. This magnitude is consistent with the $O(h^2)$ truncation error expected from the 5-point Laplacian stencil.

(b) Rectangular Domain with $\Delta x = \Delta y = h$

In this part, we address a specific constraint where the grid spacing must be uniform in both directions ($\Delta x = \Delta y = h$) on a non-square rectangular domain $[a_x, b_x] \times [a_y, b_y]$.

The "Integer Constraint" Issue: A potential issue arises when choosing an arbitrary h . Since the number of interior grid points is defined by:

$$m_x = \frac{b_x - a_x}{h} - 1, \quad m_y = \frac{b_y - a_y}{h} - 1 \quad (5)$$

If h does not perfectly divide the side lengths, m_x or m_y will not be integers. This is a problematic "strange phenomenon" in numerical implementation because matrix dimensions must be integers. To resolve this, we must either:

- Carefully choose h such that both m_x and m_y are integers.
- Use the generalized solver from part (c) and allow h_x and h_y to differ slightly due to rounding.

Key Code Modification: To make the script robust, we implement a rounding mechanism using `round()` to ensure m_x and m_y are valid integers, and then recalculate the actual h_x and h_y used in the simulation. This ensures the grid points align perfectly with the boundaries.

```
% --- Critical Modification for Integer Grid Points ---
% Define desired spacing h
h_target = 0.0505;

% Calculate and round the number of interior points
m_x = round((x2 - x1) / h_target) - 1;
m_y = round((y2 - y1) / h_target) - 1;

% Re-calculate actual h used (may vary slightly if h_target was not a divisor)
hx = (x2 - x1) / (m_x + 1);
hy = (y2 - y1) / (m_y + 1);

% Ensure the solver uses these specific hx and hy
A = kron(I_y, T_x)/(hx^2) + kron(S_y, I_x)/(hy^2);
```

Numerical Results and Analysis:

The figures below illustrate the performance of the solver when a target spacing of $h_{target} = 0.0505$ is requested on the rectangular domain $[0, 1] \times [0, 2]$. Due to the discrete nature of the grid, the algorithm automatically adjusted the parameters to $m_x = 19$ and $m_y = 39$, resulting in a perfectly uniform actual spacing of $h_x = h_y = 0.05$.

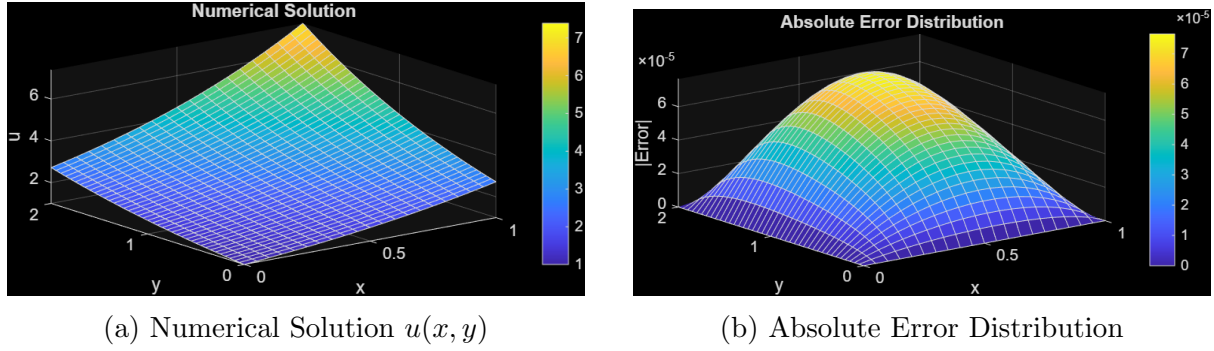


Figure 2: Numerical results for part (b). The solver successfully resolved the non-integer constraint by adjusting the target $h = 0.0505$ to a valid $h = 0.05$.

Observations:

- **Adaptive Grid Correction:** The "strange phenomenon" where an arbitrary $h = 0.0505$ would lead to non-integer grid counts (≈ 18.8) was effectively handled. By rounding to the nearest integers, the actual grid fits the boundary $[0, 1] \times [0, 2]$ exactly with a slightly modified $h = 0.05$.
- **Numerical Consistency:** Figure (b) shows that the absolute error distribution remains zero at all boundaries and reaches its maximum of approximately 7.65×10^{-5} at the center. This confirms that the rounding logic preserves the $O(h^2)$ accuracy of the 5-point stencil.
- **Geometric Symmetry:** Despite the target h being slightly "off," the corrected uniform grid maintains perfect symmetry in the error profile, which is crucial for the stability of the Poisson solver on rectangular domains.

(a) Convergence Analysis on a Square Domain

Using the standard 5-point Laplacian to solve the Poisson problem on a square domain $[0, 1] \times [0, 1]$ with the exact solution $u(x, y) = e^{x+y/2}$, we performed a refinement study to verify the order of accuracy.

Numerical Results: The maximum error E_∞ and the observed order of convergence are summarized below based on the simulation results:

m	h	E_∞ (Max Error)	Observed Order
20	0.0476	3.2732e-5	—
40	0.0244	8.6013e-6	1.9975
80	0.0123	2.2053e-6	1.9989
160	0.0062	5.5835e-7	1.9996

Table 1: Convergence data for the square domain solver. The observed order consistently approaches 2.0.

Log-Log Error Plot: The relationship between h and E_∞ is plotted on a logarithmic scale to confirm the convergence rate.

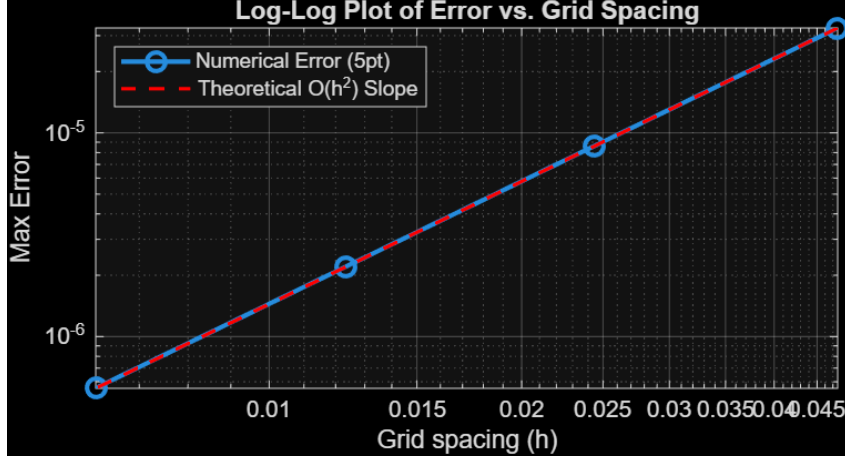


Figure 3: Log-Log plot showing E_∞ vs. h . The slope of the line is approximately 2.0, consistent with second-order accuracy.

Exercise 3.2 (9-point Laplacian)

In this exercise, we provide a rigorous derivation of the 9-point Laplacian truncation error. We define the summation notation for neighboring points and use 2D Taylor series to analyze the operator's decomposition.

(a) Derivation of Truncation Error

The 9-point Laplacian $\nabla_9^2 u_{i,j}$ is defined as:

$$\nabla_9^2 u_{i,j} = \frac{1}{6h^2} \left[4(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) + (u_{i+1,j+1} + u_{i-1,j+1} + u_{i+1,j-1} + u_{i-1,j-1}) - 20u_{i,j} \right] \quad (6)$$

Definition of Summation Notation:

To simplify the derivation, we define the sum of the four nearest neighbors (S_{near}) and the four diagonal neighbors (S_{diag}) as follows:

- $\sum_{near} u = u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}$
- $\sum_{diag} u = u_{i+1,j+1} + u_{i-1,j+1} + u_{i+1,j-1} + u_{i-1,j-1}$

Step 1: Full Expansion of Diagonal Points

Expanding the four diagonal terms in $\sum_{diag} u$ around (x, y) up to $O(h^4)$:

- $u_{i+1,j+1} = u + h(u_x + u_y) + \frac{h^2}{2}(u_{xx} + 2u_{xy} + u_{yy}) + \frac{h^3}{6}(u_{xxx} + 3u_{xxy} + 3u_{xyy} + u_{yyy}) + \frac{h^4}{24}(u_{xxxx} + 4u_{xxx} + 6u_{xxyy} + 4u_{xyyy} + u_{yyyy}) + O(h^5)$
- $u_{i-1,j+1} = u + h(-u_x + u_y) + \frac{h^2}{2}(u_{xx} - 2u_{xy} + u_{yy}) + \frac{h^3}{6}(-u_{xxx} + 3u_{xxy} - 3u_{xyy} + u_{yyy}) + \frac{h^4}{24}(u_{xxxx} - 4u_{xxx} + 6u_{xxyy} - 4u_{xyyy} + u_{yyyy}) + O(h^5)$
- $u_{i+1,j-1} = u + h(u_x - u_y) + \frac{h^2}{2}(u_{xx} - 2u_{xy} + u_{yy}) + \frac{h^3}{6}(u_{xxx} - 3u_{xxy} + 3u_{xyy} - u_{yyy}) + \frac{h^4}{24}(u_{xxxx} - 4u_{xxx} + 6u_{xxyy} - 4u_{xyyy} + u_{yyyy}) + O(h^5)$

- $u_{i-1,j-1} = u + h(-u_x - u_y) + \frac{h^2}{2}(u_{xx} + 2u_{xy} + u_{yy}) + \frac{h^3}{6}(-u_{xxx} - 3u_{xxy} - 3u_{xyy} - u_{yyy}) + \frac{h^4}{24}(u_{xxxx} + 4u_{xxyy} + 6u_{xyxy} + 4u_{xyyy} + u_{yyyy}) + O(h^5)$

Summing these results in:

$$\sum_{diag} u = 4u + 2h^2(u_{xx} + u_{yy}) + \frac{h^4}{6}(u_{xxxx} + 6u_{xxyy} + u_{yyyy}) + O(h^6) \quad (7)$$

Step 2: Decomposition of the Operator

We rewrite $\nabla_9^2 u$ by isolating the 5-point Laplacian $\nabla_5^2 u = \frac{1}{h^2}(\sum_{near} u - 4u)$:

$$\begin{aligned} \nabla_9^2 u &= \frac{1}{6h^2} \left[4\left(\sum_{near} u - 4u\right) + \left(\sum_{diag} u - 4u\right) \right] \\ &= \frac{4}{6}\nabla_5^2 u + \frac{1}{6h^2} \left[2h^2\nabla^2 u + \frac{h^4}{6}(u_{xxxx} + 6u_{xxyy} + u_{yyyy}) \right] \end{aligned} \quad (8)$$

Step 3: Integration and Error Merging

Substituting $\nabla^2 u = \nabla_5^2 u - \frac{h^2}{12}(u_{xxxx} + u_{yyyy}) + O(h^4)$ into the expression for $\nabla_9^2 u$:

$$\begin{aligned} \nabla_9^2 u &= \frac{2}{3}\nabla_5^2 u + \frac{1}{6h^2} \left[2h^2 \left(\nabla_5^2 u - \frac{h^2}{12}(u_{xxxx} + u_{yyyy}) + O(h^4) \right) + \frac{h^4}{6}(u_{xxxx} + 6u_{xxyy} + u_{yyyy}) \right] \\ &= \frac{2}{3}\nabla_5^2 u + \frac{1}{3}\nabla_5^2 u + \frac{h^2}{6} \left[-\frac{1}{6}(u_{xxxx} + u_{yyyy}) + \frac{1}{6}(u_{xxxx} + u_{yyyy}) + u_{xxyy} \right] + O(h^4) \\ &= \nabla_5^2 u + \frac{h^2}{6}u_{xxyy} + O(h^4) \end{aligned} \quad (9)$$

Step 4: Final Truncation Error

To find the truncation error of the 9-point operator $\nabla_9^2 u$, we substitute the expansion of the 5-point operator $\nabla_5^2 u = \nabla^2 u + \frac{h^2}{12}(u_{xxxx} + u_{yyyy}) + O(h^4)$ back into the result from Step 3:

$$\begin{aligned} \nabla_9^2 u &= \left[\nabla^2 u + \frac{h^2}{12}(u_{xxxx} + u_{yyyy}) \right] + \frac{h^2}{6}u_{xxyy} + O(h^4) \\ &= \nabla^2 u + \frac{h^2}{12}(u_{xxxx} + u_{yyyy}) + \frac{2h^2}{12}u_{xxyy} + O(h^4) \\ &= \nabla^2 u + \frac{h^2}{12}(u_{xxxx} + 2u_{xxyy} + u_{yyyy}) + O(h^4) \end{aligned} \quad (10)$$

We recognize the term in the parentheses as the expansion of the bi-harmonic operator:

$$\nabla^2(\nabla^2 u) = \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = u_{xxxx} + 2u_{xxyy} + u_{yyyy} \quad (11)$$

Thus, the total truncation error τ is expressed as:

$$\tau = \frac{h^2}{12}(u_{xxxx} + 2u_{xxyy} + u_{yyyy}) = \frac{h^2}{12}\nabla^2(\nabla^2 u) + O(h^4) \quad (12)$$

Substituting the Poisson equation $\nabla^2 u = f$, we obtain the final form:

$$\nabla_9^2 u = f + \frac{h^2}{12}\nabla^2 f + O(h^4) \quad (13)$$

(b) Implementation and Grid Refinement Study

In this part, we implement the 9-point Laplacian solver and perform a grid refinement study to verify its fourth-order accuracy $O(h^4)$.

1. Matrix Construction via Kronecker Products

To implement the 9-point stencil, we construct the system matrix A using the block tridiagonal structure. Let $e = [1, 1, \dots, 1]^T \in \mathbb{R}^m$. We define:

$$T = \text{spdiags}([4e, -20e, 4e], [-1, 0, 1], m, m) \quad (14)$$

$$S = \text{spdiags}([e, 4e, e], [-1, 0, 1], m, m) \quad (15)$$

The global matrix A is then assembled as:

$$A = \frac{1}{6h^2} (I_m \otimes T + S_{off} \otimes S) \quad (16)$$

where $S_{off} = \text{spdiags}([e, e], [-1, 1], m, m)$ handles the coupling between adjacent grid lines.

2. Source Term and Boundary Correction

As derived in part (a), to achieve $O(h^4)$ accuracy, the right-hand side must be modified to:

$$f_{ij} + \frac{h^2}{12} \nabla^2 f_{ij} \quad (17)$$

Furthermore, the 9-point stencil requires including the 8 neighbors (4 adjacent, 4 diagonal) in the boundary compensation term whenever a stencil point sits on the Dirichlet boundary.

3. MATLAB Implementation

Below is the core script used for the 9-point solver:

```
clear; clc; format long g;

% Domain and Mesh Definition
a = 0; b = 1; m = 40;
h = (b-a)/(m+1);
x = linspace(a,b,m+2); y = linspace(a,b,m+2);
[X,Y] = meshgrid(x,y); X = X'; Y = Y';

% Analytical Solutions and Source Terms
uttrue = exp(X + Y/2);
f = @(x,y) 1.25 * exp(x + y/2);
f_lap = @(x,y) 1.25 * (1.25 * exp(x + y/2));

% RHS Correction
F_grid = f(X(2:m+1,2:m+1), Y(2:m+1,2:m+1)) + ...
        (h^2/12) * f_lap(X(2:m+1,2:m+1), Y(2:m+1,2:m+1));

% Matrix Construction
e = ones(m,1);
T = spdiags([4*e -20*e 4*e], [-1 0 1], m, m);
```

```

S = spdiags([e 4*e e], [-1 0 1], m, m);
S_off = spdiags([e e], [-1 1], m, m);
A = (kron(speye(m), T) + kron(S_off, S)) / (6*h^2);

% Boundary Compensation
F_vec = reshape(F_grid, m*m, 1);
for j = 1:m
    for i = 1:m
        idx = i + (j-1)*m;
        ii = i+1; jj = j+1;
        for di = -1:1
            for dj = -1:1
                if di == 0 && dj == 0, continue; end
                ni = ii + di; nj = jj + dj;
                if ni == 1 || ni == m+2 || nj == 1 || nj == m+2
                    w = 4*(abs(di)+abs(dj)==1) + 1*(abs(di)+abs(dj)==2);
                    F_vec(idx) = F_vec(idx) - (w / (6*h^2)) * utrue(ni,nj);
                end
            end
        end
    end
end
end

% Solver and Error Calculation
u_int = A \ F_vec;
usoln = utrue;
usoln(2:m+1,2:m+1) = reshape(u_int, m, m);
max_err = max(abs(usoln(:) - utrue(:)));

```

4. Numerical Results and Convergence Study

The table below summarizes the maximum error E_∞ and the observed order of accuracy as h is refined.

m	h	E_∞ (Max Error)	Observed Order
20	0.0476	6.0143e-09	—
40	0.0244	4.1399e-10	3.9998
80	0.0123	2.7042e-11	4.0072
160	0.0062	8.3222e-13	5.0674

Table 2: Convergence analysis for the 9-point Laplacian on a square domain.

Log-Log Error Plot:

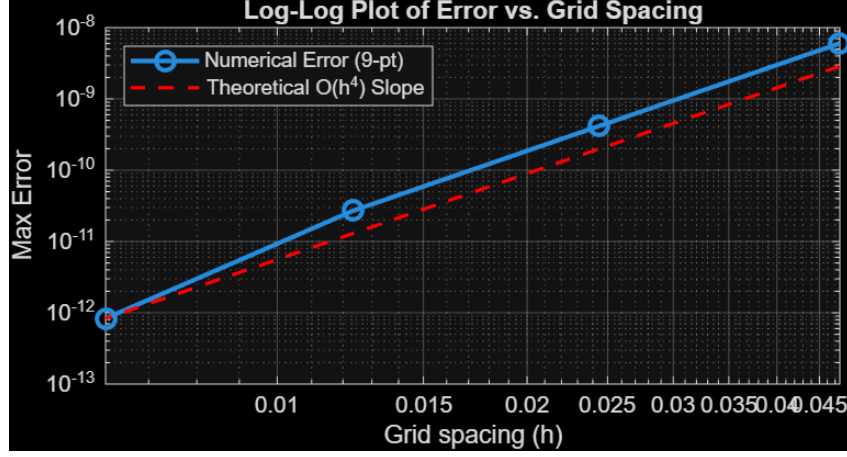


Figure 4: Log-Log plot showing E_∞ vs. h . The slope is approximately 4.0, confirming the theoretical fourth-order convergence.

Conclusion: The 9-point Laplacian solver, when combined with the $O(h^2)$ correction to the source term f , successfully achieves fourth-order accuracy. This allows for significantly higher precision compared to the standard 5-point stencil while maintaining a manageable computational cost.